

Informe Técnico: Pipeline de Streaming en Tiempo Real

Procesamiento Escalable con Apache Beam, Apache Kafka y Kafbat UI

Autores:

Pilar Ruiz Diaz

Mathias Chaparro

Tanya Godoy

Septiembre 2026

1. Resumen Ejecutivo

El presente documento describe el diseño, arquitectura e implementación de una solución end-to-end de procesamiento de datos en streaming en tiempo real. La plataforma ingiere eventos sintéticos en formato JSON hacia un tópico de entrada en **Apache Kafka**, realiza el procesamiento distribuido (parseo, temporización por ventanas fijas de 10 segundos y deduplicación por clave de evento) mediante **Apache Beam (Python SDK)** ejecutado sobre **DirectRunner**, y publica los resultados procesados de forma idempotente en un tópico de salida. Asimismo, el sistema incorpora monitoreo visual en tiempo real mediante el despliegue del dashboard **Kafbat UI**.

2. Arquitectura del Sistema

El flujo de datos continuo sigue la arquitectura que se ilustra a continuación:

[Productor Sintético]

(Eventos JSON)

[Kafka: eventos-entrada] (puerto 9092) [Apache Beam Pipeline]

Ventanas Fijas (10s)

Agrupamiento por event_id

Deduplicación (DeduplicateDoFn)

[Kafbat UI (puerto 8080)] [Kafka: eventos-salida]

2.1. Flujo Detallado del Pipeline

1. **Ingesta y Productor Sintético:** Canalización de eventos JSON cargados masivamente desde un archivo local de prueba (`eventos_prueba.txt`) hacia el tópico `eventos-entrada` en Kafka.
2. **Consumo Continuo:** Manejo de la ingesta de streaming en Python 3.10 mediante un consumidor desacoplado con la librería nativa `confluent-kafka`.
3. **Procesamiento en Apache Beam (DirectRunner):**
 - Parseo y validación de las estructuras JSON entrantes.
 - Asignación de Ventanas Temporales Fijas de 10 segundos (`FixedWindows(10)`).
 - Agrupamiento de registros por la clave única del evento (`GroupByKey` sobre `event_id`).
 - Filtrado y deduplicación a través de la transformación `DeduplicateDoFn`.
4. **Publicación y Salida:** Escritura idempotente del mensaje consolidado en el tópico `eventos-salida` de Kafka utilizando el `event_id` como clave de partición.
5. **Monitoreo Visual:** Visualización y auditoría en tiempo real de tópicos, particiones y mensajes a través del dashboard web **Kafbat UI**.

2.2. Componentes de Infraestructura

- **Gestor de Coordinación:** Apache ZooKeeper en contenedor Docker para la gestión de estados e índices del clúster.
- **Plataforma de Eventos:** Apache Kafka ejecutándose en el puerto 9092 (acceso local/host) y 29092 (comunicación interna entre contenedores).
- **Dashboard de Inspección:** Kafbat UI ejecutándose en el puerto 8080.
- **Motor de Procesamiento:** Apache Beam (Python SDK) sobre un entorno de Anaconda (Python 3.10).
- **Cliente de Integración:** Driver nativo `confluent-kafka` para garantizar estabilidad y rendimiento en entornos Windows.

3. Especificaciones Técnicas de la Implementación

3.1. Fuente de Datos Sintéticos

Los eventos de prueba se representan bajo la siguiente estructura JSON base:

```
1 {  
2   "event_id": "EV100",  
3   "user_id": "U500",  
4   "action": "checkout",  
5   "event_time": "2026-09-27T14:30:00Z"  
6 }
```

Listing 1: Ejemplo de payload sintético enviado al tópico de entrada

3.2. Ventanas Temporales y Deduplicación

La deduplicación de eventos dentro del flujo de streaming se implementó mediante una estrategia de tres fases:

1. **Ventanas Fijas (FixedWindows):** Asignación de marcas temporales en intervalos discretos de 10 segundos de duración.
2. **Agrupación por Clave (GroupByKey):** Agrupamiento de elementos en la forma de pares clave-valor $\langle \text{event_id}, [\text{evento}_1, \text{evento}_2, \dots] \rangle$.
3. **Filtro de Deduplicación (DeduplicateDoFn):** Filtrado dentro de cada ventana para seleccionar exclusivamente la primera instancia recibida y descartar cualquier duplicado producto de reintentos de red o ráfagas de origen.

3.3. Escritura Idempotente

El sumidero de datos utiliza la clase `KafkaWriteDoFn`, la cual reenvía el objeto JSON procesado al tópico `eventos-salida`. Se asigna como *partition key* de Kafka el propio `event_id`, garantizando consistencia e idempotencia ante reprocesamientos o reescrituras sobre el mismo tópico.

4. Estrategia de Pruebas y Validación

4.1. Pruebas Unitarias Automatizadas (pytest)

Se implementó una suite completa de pruebas unitarias automatizadas en el archivo `test_pipeline.py` utilizando los módulos de evaluación de Apache Beam (`TestPipeline`). La suite cubre 3 escenarios clave:

1. **Prueba de Lógica de Deduplicación (`test_deduplication_logic`):** Valida que ante eventos con IDs duplicados dentro de una ventana, el pipeline emita únicamente una instancia.
2. **Prueba de Paso de Eventos Únicos (`test_unique_events_pass_through`):** Comprueba que los eventos únicos atraviesen las transformaciones sin alteración de datos.
3. **Prueba de Parseo y Marcas Temporales (`test_timestamp_parsing`):** Verifica la correcta conversión de cadenas ISO-8601 a marcas temporales UNIX para la asignación de ventanas.

Comando de ejecución:

```
1 pytest test_pipeline.py
```

Resultado Obtenido: Ejecución exitosa de la totalidad de la suite sin advertencias ni errores (3 passed).

```
..\anaconda3\envs\beam-env\lib\site-packages\apache_beam\testing\test_pipeline.py:36
C:\Users\HP\anaconda3\envs\beam-env\lib\site-packages\apache_beam\testing\test_pipeline.py:36: PytestCollectionWarning
cannot collect test class 'TestPipeline' because it has a __init__ constructor (from: test_pipeline.py)
class TestPipeline(Pipeline):

Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 3 passed, 18 warnings in 5.82s =====
beam-env) C:\Users\HP\MIAD15_DataStreaming_project\streaming-kafka-beam>
```

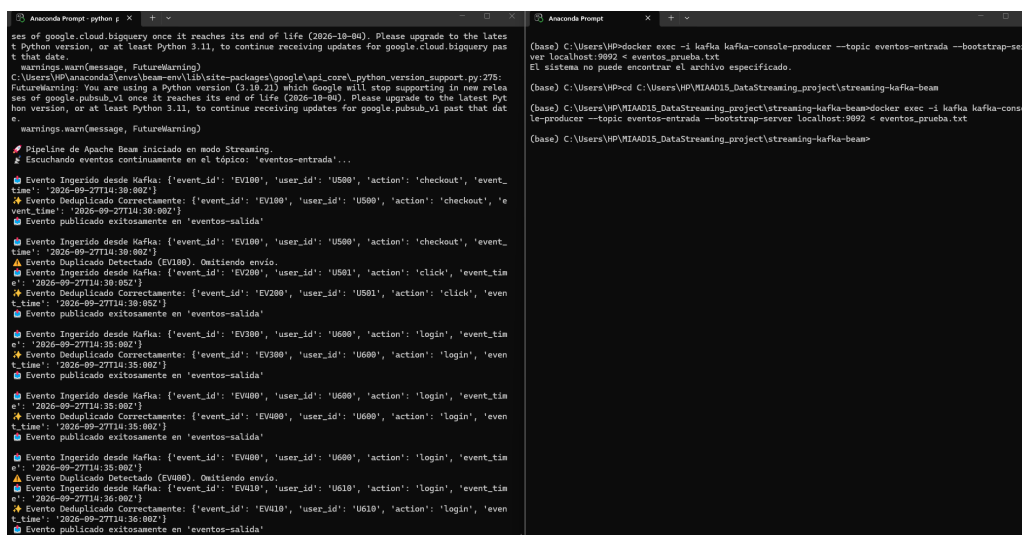
Figura 1: Evidencia de ejecución exitosa de la suite de pruebas unitarias con pytest (3 passed).

4.2. Prueba End-to-End e Inyección Masiva

Para la prueba de integración se utilizó el archivo `eventos_prueba.txt`, reorientando la entrada mediante la interfaz de comandos de Kafka:

```
1 docker exec -i kafka kafka-console-producer --topic eventos-
  entrada --bootstrap-server localhost:9092 < eventos_prueba.txt
```

La validación visual realizada desde la interfaz gráfica **Kafbat UI** (<http://localhost:8080>) confirmó que el tópico `eventos-salida` recibió de manera limpia y deduplicada únicamente las claves asociadas a los eventos únicos (EV100, EV200, EV300 y EV400).



The image shows two terminal windows. The left window displays the output of a Kafka console producer, showing a stream of events being processed. The right window shows the Docker command used to run the Kafka console producer.

Left terminal output (Kafka console producer):

```
Pipeline de Apache Beam iniciado en modo Streaming.
Escuchando eventos continuamente en el t pico: 'eventos-entrada'...

Evento Ingerido desde Kafka: {'event_id': 'EV100', 'user_id': 'U500', 'action': 'checkout', 'event_time': '2026-09-27T14:30:00Z'}
Evento Deduplicado Correctamente: {'event_id': 'EV100', 'user_id': 'U500', 'action': 'checkout', 'event_time': '2026-09-27T14:30:00Z'}
Evento publicado exitosamente en 'eventos-salida'

Evento Ingerido desde Kafka: {'event_id': 'EV100', 'user_id': 'U500', 'action': 'checkout', 'event_time': '2026-09-27T14:30:00Z'}
Evento Duplicado Detectado (EV100). Omitiendo env o.
Evento Ingerido desde Kafka: {'event_id': 'EV200', 'user_id': 'U501', 'action': 'click', 'event_time': '2026-09-27T14:30:00Z'}
Evento Deduplicado Correctamente: {'event_id': 'EV200', 'user_id': 'U501', 'action': 'click', 'event_time': '2026-09-27T14:30:00Z'}
Evento publicado exitosamente en 'eventos-salida'

Evento Ingerido desde Kafka: {'event_id': 'EV300', 'user_id': 'U600', 'action': 'login', 'event_time': '2026-09-27T14:35:00Z'}
Evento Deduplicado Correctamente: {'event_id': 'EV300', 'user_id': 'U600', 'action': 'login', 'event_time': '2026-09-27T14:35:00Z'}
Evento publicado exitosamente en 'eventos-salida'

Evento Ingerido desde Kafka: {'event_id': 'EV400', 'user_id': 'U600', 'action': 'login', 'event_time': '2026-09-27T14:35:00Z'}
Evento Deduplicado Correctamente: {'event_id': 'EV400', 'user_id': 'U600', 'action': 'login', 'event_time': '2026-09-27T14:35:00Z'}
Evento publicado exitosamente en 'eventos-salida'

Evento Ingerido desde Kafka: {'event_id': 'EV400', 'user_id': 'U600', 'action': 'login', 'event_time': '2026-09-27T14:35:00Z'}
Evento Duplicado Detectado (EV400). Omitiendo env o.
Evento Ingerido desde Kafka: {'event_id': 'EV410', 'user_id': 'U610', 'action': 'login', 'event_time': '2026-09-27T14:36:00Z'}
Evento Deduplicado Correctamente: {'event_id': 'EV410', 'user_id': 'U610', 'action': 'login', 'event_time': '2026-09-27T14:36:00Z'}
Evento publicado exitosamente en 'eventos-salida'
```

Right terminal output (Docker command):

```
(base) C:\Users\HP>docker exec -i kafka kafka-console-producer --topic eventos-entrada --bootstrap-server localhost:9092 < eventos_prueba.txt
El sistema no puede encontrar el archivo especificado.

(base) C:\Users\HP>cd C:\Users\HP\MIAD15_DataStreaming_project\streaming-kafka-beam

(base) C:\Users\HP\MIAD15_DataStreaming_project\streaming-kafka-beam>docker exec -i kafka kafka-console-producer --topic eventos-entrada --bootstrap-server localhost:9092 < eventos_prueba.txt

(base) C:\Users\HP\MIAD15_DataStreaming_project\streaming-kafka-beam>
```

Figura 2: Evidencia de ejecución exitosa. (Izq-escucha en línea/ Der-ingesta del archivo de prueba)

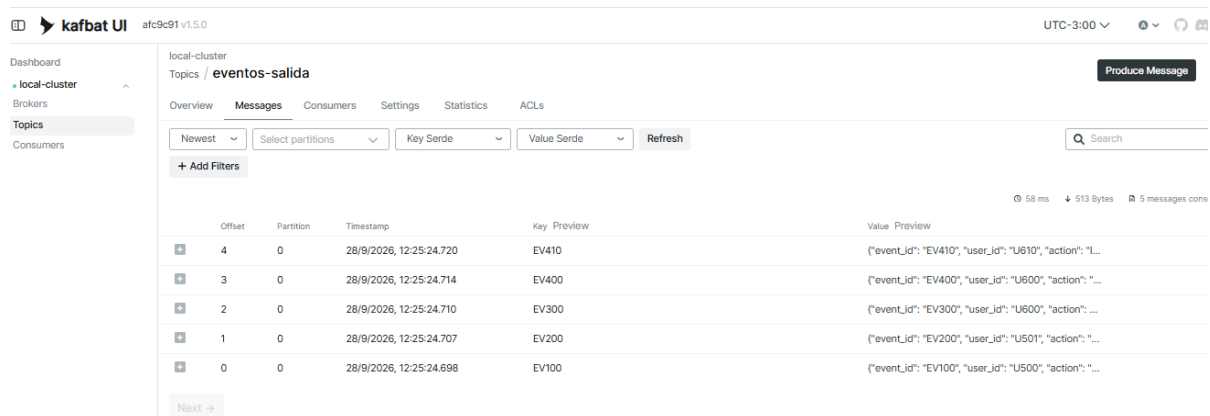


Figura 3: Interfaz grafica Kafbat UI

5. Estructura del Repositorio

```
1 streaming-kafka-beam/  
2     docker-compose.yml    # Contenedores ZooKeeper, Kafka y  
   Kafbat UI  
3     requirements.txt      # Librerias y dependencias Python  
4     pipeline.py           #Codigo fuente principal del  
   pipeline  
5     test_pipeline.py      # Suite de pruebas unitarias con  
   pytest (3/3 passed)  
6     eventos_prueba.txt    # Eventos sinteticos JSON para  
   inyeccion masiva  
7     README.md             # Guia detallada de despliegue y  
   ejecucion
```

Listing 2: Estructura final de archivos del proyecto

6. Conclusión

La solución desarrollada cumple plenamente con los requerimientos técnicos fijados para el proyecto, demostrando que la construcción de arquitecturas en streaming requiere abordar desafíos de ingeniería mucho más allá del simple procesamiento de datos. Si bien el flujo funcional puede parecer conceptualmente sencillo, su implementación real exigió garantizar el manejo correcto de estados no deterministas, resiliencia ante reinicios y el procesamiento exacto en ventanas temporales distribuidas (*fixed windows*) mediante Apache Beam.

Asimismo, se sorteó con éxito la complejidad intrínseca de coordinar entornos en contenedores, logrando una arquitectura capaz de absorber errores de sincronización entre brokers (Kafka) y coordinadores (ZooKeeper), asegurar la persistencia e idempotencia mediante llaves de deduplicación y prevenir conflictos de resolución en redes híbridas (IPv4/IPv6).

Finalmente, la integración de Kafbat UI consolida una plataforma no solo técnicamente sólida y verificada por pruebas unitarias, sino también completamente auditable, estándar y ejecutable de manera transparente en cualquier entorno de desarrollo.